Scalability & Future Plan

Date	06 October 2026
Team ID	UNM-293 2026-2430
Project Name	FitBuddy-AI Fitness Plan Generator using Gemini Models
Maximum Marks	1 Mark

Scalability & Future Plan:

This document describes how FitBuddy can scale beyond its current FastAPI, Gemini, SQLite and Jinja2-based implementation. The plan focuses on handling more users and stored plans while improving performance, security, reliability and future feature growth. FitBuddy currently generates personalized 7-day workout plans, nutrition/recovery tips and feedback-based plan updates.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	SQLite is suitable for the current/local deployment but is not ideal for a large concurrent user base.	Database contention and slower access may appear as users and stored plans increase.	High
2	Gemini API calls depend on an external AI service.	Plan generation or feedback updates can be delayed by API latency or service limits.	High
3	The documented deployment is primarily local using Uvicorn and environment-based API configuration.	Production-scale availability, monitoring and fault tolerance need additional infrastructure.	Medium

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	**User Load**	FastAPI application designed for local/web interaction.	Deploy behind a production ASGI setup with load balancing and horizontal scaling for increased concurrent users.
2	**Data Storage**	SQLite with SQLAlchemy stores user information and original/updated plans.	Migrate to PostgreSQL for production-scale storage while retaining SQLAlchemy.
3	**Performance**	Gemini Pro generates workout plans and Gemini Flash generates quick nutrition tips.	Use asynchronous request handling, caching where appropriate, background processing for non-critical tasks, and efficient API calls.
4	**Security**	Gemini API key is configured through environment variables; user data is stored in the database.	Add authentication/authorization, stronger secret management, input validation, HTTPS, logging and access controls.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 1	Production-ready deployment, improved validation, authentication and monitoring.	1–2 months	More reliable and secure user experience.

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Migrate SQLite to PostgreSQL and optimize database access for larger user volumes.	2–3 months	Better scalability and concurrent data access.
Phase 3	Add progress tracking, plan history, reminders and improved feedback analytics.	3–5 months	Higher engagement and more adaptive fitness support.
Phase 4	Cloud deployment with automated scaling, observability and future mobile/API integrations.	5–8 months	Broader availability and long-term platform growth.

Work Assignment:

Team Member	Assigned Work
Kaviya sree.T	Complete the FitBuddy project implementation and integration: Gemini model selection and AI integration; FastAPI backend and routes; workout, nutrition and feedback-based plan generation; SQLite/SQLAlchemy data storage; Jinja2 frontend templates; local deployment and testing; and preparation of the scalability and future roadmap. This covers the documented milestones from model selection and architecture through deployment.

Project Basis

The uploaded FitBuddy project document describes Gemini 1.5 Pro for workout generation and plan updates, Gemini Flash for nutrition tips, FastAPI for backend routing, SQLite with SQLAlchemy for persistence, Jinja2 templates for the frontend, and local deployment with Uvicorn. Its workflow contains five milestones covering model selection and architecture, core functionality, routes.py development, frontend development and deployment.